kb2040ctl

Command reference for the kb2040-single-key colour-tap keyboard

Version 0.1.0

Adafruit KB2040 · CircuitPython

github.com/JeremyProffittOrg/kb2040-single-key

About this manual

`kb2040ctl` configures a **kb2040-single-key** board: an Adafruit KB2040 with one mechanical switch and a chain of WS2812 RGB LEDs, which enumerates as a USB keyboard *and* a second USB serial port used only for configuration.

The tool runs on **Windows, macOS and Linux**, on both x86-64 and arm64. It is a single static binary with no runtime to install. Where the three platforms genuinely differ — how serial ports are named, what permissions are needed, how a downloaded binary is unblocked — this manual says so; everywhere else the commands are identical.

The binary format and serial protocol are specified separately in `docs/format.md`; the hardware, wiring and firmware are covered in `README.md`.

Verified against an Adafruit KB2040 running CircuitPython 10.2.1.

The colour tap, in one page

A single key can only do one thing — unless time and colour become extra inputs.

Tap the key (press and release quickly) and the profile's *tap* binding fires.

Hold it and, once past the tap window, the LEDs cycle through colours one per second. Each colour is a **slot**. Release while a colour is showing and *that* slot fires.

```
press
  |<-- tap window -->|<-- slot 0 --->|<-- slot 1 --->|<-- slot 2 --->|
  |     250 ms       |    1000 ms    |    1000 ms    |    1000 ms    |
  |     (white)      |     (red)     |   (orange)    |   (yellow)    |
  0                 250            1250            2250            3250 ms

  release here...    ...or here      ...or here      ...or here
  -> tap binding     -> slot 0       -> slot 1       -> slot 2
```

The LED is the entire user interface: hold, watch for the colour you want, release.

`tap_max_ms` (default 250) and `dwell_ms` (default 1000) are per-profile settings.

Past the last colour

The per-profile `overflow` setting decides what a long hold does:

`overflow`	behaviour
`wrap`	back to the first colour, cycling forever. Overshoot is recoverable — keep holding.
`wrap_cancel`	as `wrap`, but with one **dark** slot per rotation. Release while the LEDs are off and nothing fires. This is the escape hatch.
`clamp`	stay on the last colour indefinitely. Forgiving of long holds, but an earlier slot can only be reached by releasing and starting again.

Bindings and steps

Every binding — the tap and each colour slot — is a **sequence** of steps, run in order:

step	example	effect
key	`{ "key": "M", "mods": ["CTRL", "SHIFT"] }`	press with modifiers, then release
text	`{ "text": "On my way." }`	type the string
media	`{ "consumer": "PLAY_PAUSE" }`	a Consumer Control usage
delay	`{ "delay_ms": 200 }`	wait before the next step

Profiles

The board stores several complete configurations, one active at a time. The factory configuration ships with two.

Profile 0, `colors` (active) is self-describing — a tap sends Print Screen, and each colour slot types the name of the colour it is showing:

Gesture	Colour	Action
tap	white	Print Screen
slot 0	`#FF0000`	types `Red`
slot 1	`#FF6000`	types `Orange`
slot 2	`#FFC000`	types `Yellow`
slot 3	`#00FF00`	types `Green`
slot 4	`#00FFFF`	types `Cyan`
slot 5	`#0000FF`	types `Blue`
slot 6	`#8000FF`	types `Violet`
slot 7	`#FF00FF`	types `Magenta`

Profile 1, `media` is play/pause on tap, with mute, volume and track controls on the colour slots, and a final slot that screenshots, waits, and pastes.

Installing

Download the binary for your platform from the releases page, or build from source. There is no runtime dependency either way.

Platform	Release asset
Windows (Intel/AMD)	`kb2040ctl_<version>_windows_amd64.exe`
Windows (Arm)	`kb2040ctl_<version>_windows_arm64.exe`
macOS (Apple silicon)	`kb2040ctl_<version>_darwin_arm64`
macOS (Intel)	`kb2040ctl_<version>_darwin_amd64`
Linux (Intel/AMD)	`kb2040ctl_<version>_linux_amd64`
Linux (Arm, e.g. Raspberry Pi)	`kb2040ctl_<version>_linux_arm64`

`SHA256SUMS` is published alongside them.

Windows

Rename the download to `kb2040ctl.exe` and put it somewhere on your `PATH`. Nothing else is required: Windows 10 and 11 bind USB CDC devices to the built-in `usbser.sys` driver, so the board needs no driver install.

SmartScreen may warn that the publisher is unknown, because the binary is not code-signed. Choose *More info → Run anyway*, or unblock it once in PowerShell:

```
Unblock-File .\kb2040ctl.exe
```

macOS

Make it executable, and clear the quarantine flag that Gatekeeper puts on anything downloaded from a browser:

```
chmod +x kb2040ctl_0.1.0_darwin_arm64
xattr -d com.apple.quarantine kb2040ctl_0.1.0_darwin_arm64
sudo mv kb2040ctl_0.1.0_darwin_arm64 /usr/local/bin/kb2040ctl
```

Without the `xattr` step macOS refuses to run it — "*cannot be opened because the developer cannot be verified*". No driver or extension is needed; serial access requires no special permission.

Linux

```
chmod +x kb2040ctl_0.1.0_linux_amd64
sudo mv kb2040ctl_0.1.0_linux_amd64 /usr/local/bin/kb2040ctl
```

Serial port permissions are the one extra step. On most distributions `/dev/ttyACM*` is owned by root and the `dialout` group, so a normal user gets *permission denied* until they join it:

```
sudo usermod -aG dialout "$USER"      # Debian, Ubuntu, Raspberry Pi OS
sudo usermod -aG uucp "$USER"         # Arch, and some Fedora setups
```

Log out and back in for the new group to apply — `newgrp dialout` works for the current shell in the meantime. Check with `groups`.

If you would rather not add a group membership, a udev rule scoped to Adafruit's vendor ID does the same job and also stops ModemManager from interfering (see *Troubleshooting*):

```
# /etc/udev/rules.d/99-kb2040.rules
SUBSYSTEM=="tty", ATTRS{idVendor}=="239a", MODE="0660", GROUP="plugdev", \
  ENV{ID_MM_DEVICE_IGNORE}="1"
```

```
sudo udevadm control --reload-rules && sudo udevadm trigger
```

Building from source

Requires Go 1.24 or newer. No cgo, so cross-compiling needs nothing beyond the toolchain:

```
go build -o kb2040ctl ./cli/cmd/kb2040ctl
```

On macOS a native build (cgo enabled by default) additionally reads USB descriptors through IOKit, so `kb2040ctl ports` can show vendor and product IDs. The released macOS binaries are built without cgo and show port names only; this affects the display, not the ability to find the board.

Getting the firmware onto the board

Covered fully in `README.md`. In short, once the board mounts as `CIRCUITPY`:

```
pwsh scripts/flash.ps1        # Windows
```

```
scripts/flash.sh              # macOS and Linux
```

Both install the CircuitPython libraries with `circup` and copy `src/` onto the board. Then **unplug and replug** — `boot.py` only takes effect on a hard reset, and it is what creates the configuration serial port.

Using the CLI

Synopsis

```
kb2040ctl <command> [flags] [arguments]
```

Run `kb2040ctl` with no arguments for a summary, or `kb2040ctl <command> -h` for one command's flags.

Finding the board

Every command that talks to the board accepts `-port`:

```
kb2040ctl info -port COM28                    # Windows
kb2040ctl info -port /dev/cu.usbmodem1101     # macOS
kb2040ctl info -port /dev/ttyACM1             # Linux
```

Without it, the board is found by **probing**: each candidate serial port is asked for its version, and the one that answers is the board.

This is deliberate rather than matching a USB ID. The board exposes *two* CDC ports — the CircuitPython REPL console and the configuration port — with identical vendor and product IDs. Only the second answers this protocol, so the ID cannot tell them apart. Probing also means a board with customised USB identification still works.

What the two ports look like

Platform	Typical names	Notes
Windows	`COM27`, `COM28`	numbers are assigned by Windows and are not predictable
macOS	`/dev/cu.usbmodem1101`, `/dev/cu.usbmodem1103`	use the `cu.` names, never `tty.`
Linux	`/dev/ttyACM0`, `/dev/ttyACM1`	numbering depends on what else is plugged in

Of each pair the lower is normally the REPL console and the higher the configuration port, but do not rely on it — `kb2040ctl ports` marks the right one.

On macOS, only `/dev/cu.*` ports are considered. Opening the `/dev/tty.*` twin of a callout device blocks until carrier detect is asserted, which never happens for a USB CDC port, so probing it would hang. Bluetooth ports are skipped on every platform for the same class of reason — on Windows, opening an idle Bluetooth serial port can block for minutes.

Exit status

Status	Meaning
0	success
1	the command failed; the reason is printed to standard error
2	the command was not recognised, or its arguments were wrong

Machine-readable output (`download` , `get`) goes to standard output; progress and diagnostics go to standard error, so `kb2040ctl download > config.json` is safe on every platform.

Commands

ports

```
kb2040ctl ports
```

Lists every serial port and marks the board with `->`.

```
$ kb2040ctl ports                    # Windows
   COM27        USB Serial Device (COM27)  [USB 239A:8106]
-> COM28        USB Serial Device (COM28)  [USB 239A:8106]
   COM3         Standard Serial over Bluetooth link (COM3)
```

```
$ kb2040ctl ports                    # Linux
   /dev/ttyACM0   KB2040 Single Key  [USB 239a:8106]
-> /dev/ttyACM1   KB2040 Single Key  [USB 239a:8106]
```

```
$ kb2040ctl ports                    # macOS (released build, no cgo)
   /dev/cu.usbmodem1101   (no product name)
-> /dev/cu.usbmodem1103   (no product name)
```

If no board answers, the reason is printed after the list.

info

```
kb2040ctl info [-port PORT]
```

Firmware version, storage use and the profile list. The active profile is marked `*`.

```
$ kb2040ctl info
port       COM28
firmware   0.1.0 (format 1)
storage    204 / 4096 bytes used (3892 free)

* 0  colors           8 slots, 1000ms dwell, wrap overflow, 8 LEDs
  1  media            6 slots, 1000ms dwell, wrap_cancel overflow, 8 LEDs
```

A `status` line appears only when the board could **not** read its stored configuration and fell back to the factory defaults:

status	meaning
`blank`	nothing has ever been written to NVM
`corrupt: …`	something was there but failed to decode; the reason follows
`no-nvm`	the board has no non-volatile memory; configuration is RAM-only

Uploading any configuration clears it.

download

```
kb2040ctl download [-port PORT] [-p N] [-o FILE]
```

Saves the board's configuration as JSON. Writes to standard output unless `-o` is given.

Flag	Meaning
`-p N`	download only profile *N* rather than the whole device
`-o FILE`	write to a file instead of standard output

```
$ kb2040ctl download -o mine.json
wrote mine.json (3061 bytes of JSON)

$ kb2040ctl download -p 0 -o colors.json
```

A single-profile file is the right input for `upload -p N`.

upload

```
kb2040ctl upload [-port PORT] [-p N] FILE
```

Sends a configuration to the board.

Flag	Meaning
`-p N`	replace only profile *N*, leaving the others untouched

```
$ kb2040ctl upload mine.json
uploaded 204 bytes (204 / 4096 used, 3892 free)

$ kb2040ctl upload -p 1 examples/meetings.json
```

The file is validated and the byte budget checked **before** anything is sent, and the board checks both again before committing. A configuration that does not fit is refused at both ends and the board is left exactly as it was.

Uploads are atomic. The transfer is buffered and checked in order — character count, decode, byte count, transport checksum, then configuration decode — and only a configuration that passes every check is stored.

get

```
kb2040ctl get [-port PORT] PATH
```

Prints one value from the board's configuration as JSON.

Paths are dot-separated; a numeric segment indexes a list.

```
$ kb2040ctl get profiles.0.dwell_ms
1000
$ kb2040ctl get profiles.0.slots.3.color
"#00FF00"
$ kb2040ctl get profiles.0.tap.steps.0
{"key":"PRINT_SCREEN"}
```

set

```
kb2040ctl set [-port PORT] PATH VALUE
```

Changes one value and uploads the result.

```
$ kb2040ctl set profiles.0.dwell_ms 1500
$ kb2040ctl set profiles.0.slots.2.color "#00FF80"
$ kb2040ctl set profiles.0.slots.0.steps.0.text "Back in five."
$ kb2040ctl set profiles.0.overflow clamp
$ kb2040ctl set active 1
```

`set` downloads the configuration, applies the edit, validates it and uploads the whole thing. The board has no field-level write on purpose: the configuration is only ever stored as a complete, checksummed unit, so a half-applied configuration cannot exist.

The type already stored at the path decides how the argument is read. A name of `2024` stays the string `"2024"`; `dwell_ms 1500` becomes the number 1500; `quick` where a number belongs is rejected. Lists and objects are given as JSON:

```
$ kb2040ctl set profiles.0.tap.steps '[{"key":"F5"},{"delay_ms":50}]'
```

In PowerShell, single quotes work the same way; `cmd.exe` needs double quotes with the inner ones escaped, so a JSON value is easier from PowerShell.

Errors name what is available:

```
$ kb2040ctl set profiles.0.dwel_ms 1500
kb2040ctl set: profiles.0 has no field "dwel_ms"; it has: brightness, dwell_ms,
ext_count, idle_color, idle_mode, name, overflow, slots, tap, tap_max_ms
```

profile

```
kb2040ctl profile [-port PORT] list
kb2040ctl profile [-port PORT] use N
```

Lists the profiles, or switches the active one. Switching is persistent.

```
$ kb2040ctl profile use 1
active profile is now 1
```

test

```
kb2040ctl test [-port PORT] PROFILE BINDING
```

Fires a binding immediately, without touching the key. Binding 0 is the tap; 1 upwards are the colour slots.

```
$ kb2040ctl test 0 4
fired 1 step(s)
```

This sends **real keystrokes to whatever window has focus.** A text binding will type into your terminal if that is where the cursor is. Use it to separate a wiring or timing problem from a configuration problem, with the cursor somewhere harmless.

watch

```
kb2040ctl watch [-port PORT]
```

Streams key and colour-slot events live. Ctrl-C stops it.

```
$ kb2040ctl watch
Watching. Tap and hold the key; press Ctrl-C to stop.
press
slot 0 FF0000
slot 1 FF6000
slot 2 FFC000
release
fire 3
```

Event	Meaning
`press`	key went down
`slot N RRGGBB`	the hold advanced to slot *N*, showing that colour
`cancel`	the hold reached the dark `wrap_cancel` slot
`release`	key came up
`fire N`	binding *N* ran
`error …`	a binding raised; the configuration port stays alive

This is the tool for tuning timing. Overshooting slots means `dwell_ms` is too short; a deliberate tap landing on slot 0 means `tap_max_ms` is too short.

validate

```
kb2040ctl validate [-p] FILE
```

Checks a configuration file and reports the byte budget. **No board required** — useful in an editor loop or in CI.

Flag	Meaning
`-p`	the file holds a single profile rather than a whole device configuration

```
$ kb2040ctl validate examples/default.json
examples/default.json is valid: 2 profile(s), 204 / 4096 bytes (3892 free)
  0  colors           8 slots, 115 bytes
  1  media            6 slots, 73 bytes
```

defaults

```
kb2040ctl defaults [-port PORT] [-y]
```

Restores the factory configuration, replacing **every** profile. Asks for confirmation unless `-y` is given. Download a copy first if the current configuration is worth keeping.

keys

```
kb2040ctl keys [media]
```

Lists every key name a configuration may use, or with `media`, every Consumer Control name. Needs no board.

version

```
kb2040ctl version
```

Prints the tool's own version.

Configuration file format

```
{
  "format": 1,
  "active": 0,
  "profiles": [
    {
      "name": "colors",
      "dwell_ms": 1000,
      "tap_max_ms": 250,
      "overflow": "wrap",
      "ext_count": 8,
      "brightness": 64,
      "idle_mode": "breathe",
      "idle_color": "#001018",
      "tap":   { "color": "#FFFFFF", "steps": [ { "key": "PRINT_SCREEN" } ] },
      "slots": [
        { "color": "#FF0000", "steps": [ { "text": "Red" } ] },
        { "color": "#FF6000", "steps": [ { "text": "Orange" } ] }
      ]
    }
  ]
}
```

Files are plain UTF-8 JSON and move between platforms unchanged; line endings do not matter.

Profile fields

Field	Range	Meaning
`name`	≤ 16 bytes	shown by `info` and `profile list`
`dwell_ms`	100–10000	how long each colour is displayed
`tap_max_ms`	20–2000	release under this counts as a tap
`overflow`	`wrap`, `wrap_cancel`, `clamp`	behaviour past the last colour
`ext_count`	0–64	external WS2812 chain length; 0 disables it
`brightness`	0–255	applies to every LED
`idle_mode`	`off`, `solid`, `breathe`, `rainbow`	animation when the key is idle
`idle_color`	`#RRGGBB`	used by `solid` and `breathe`
`tap`	binding	fired by a quick tap
`slots`	1–16 bindings	the colour slots, in cycle order

Up to **8** profiles, subject to the byte budget.

Binding and step fields

A binding is `{ "color": "#RRGGBB", "steps": [ … ] }`. A step sets **exactly one** of:

Field	Type	Notes
`key`	name	with optional `mods`
`text`	string	1–255 bytes
`consumer`	name	Consumer Control usage
`delay_ms`	0–10000	wait

`mods` accompanies `key` only and accepts `CTRL`, `SHIFT`, `ALT`, `GUI`, and the `R`-prefixed right-hand forms. `WIN` and `CMD` are accepted as aliases for `GUI`, which is the same HID modifier bit on every platform — the host decides what it means. Run `kb2040ctl keys` for key names and `kb2040ctl keys media` for media names. An unrecognised name is rejected with a list of valid ones. A numeric form such as `0x68` is also accepted for a raw HID usage ID.

Up to 64 steps per binding.

Keystrokes are sent as HID usage codes, so what a key produces depends on the **host's** keyboard layout. Text steps are typed via a US layout; on a host set to another layout, punctuation in a text step may not come out as written.

The byte budget

The whole configuration lives in the RP2040's **4096-byte** non-volatile region. That is what lets it persist while the `CIRCUITPY` drive stays writable from your computer.

There is no fixed per-profile limit — spend the space as you like. As a rule of thumb, a slot with a key or media action costs about 8 bytes, and a slot that types a sentence costs about 6 bytes plus the length of the text. The two-profile factory configuration uses 204 bytes, leaving room for roughly thirty more profiles of that size.

`validate` and `upload` both report usage, and an oversized configuration is refused before anything is sent.

Troubleshooting

Any platform

`no kb2040-single-key found`. Either the board is not running this firmware, or another program already holds the port — a serial monitor, the Mu editor, the Arduino IDE. Close it, or pass `-port` explicitly.

Only one serial port appears. `boot.py` has not run. It takes effect only on a *hard* reset: unplug and replug, or press the reset button. A soft reload is not enough.

`info` shows a `status` that is not `ok`. The board could not read its stored configuration and is running the factory defaults. `blank` is normal on a board that has never been configured. Uploading any configuration clears it.

The LEDs work but the key does nothing. The switch is not reaching `D4` / `GND`. Run `kb2040ctl watch` — no `press` line means the firmware never sees the key.

The key fires the wrong slot. Run `kb2040ctl watch` and hold the key. If the slots advance faster than you can react, raise `dwell_ms`; if a quick tap registers as slot 0, raise `tap_max_ms`.

Nothing at all on the LEDs at startup. The firmware is not running. Check the REPL on the *console* serial port for a traceback — a missing library is the usual cause.

Windows

SmartScreen blocks the binary. It is not code-signed. *More info* → *Run anyway*, or `Unblock-File .\kb2040ctl.exe`.

A command hangs for a long time. Older builds probed every serial port, including Bluetooth ones, which can block for minutes. Current builds probe only USB ports; if you are on an older binary, pass `-port` explicitly.

macOS

"cannot be opened because the developer cannot be verified". Gatekeeper quarantine. Clear it with `xattr -d com.apple.quarantine ./kb2040ctl`.

`permission denied` when running it. Missing the executable bit: `chmod +x ./kb2040ctl`.

A command hangs. Check you are not pointing `-port` at a `/dev/tty.*` name. Use the `/dev/cu.*` twin; the `tty.` variant waits for carrier detect that a USB CDC port never asserts. Autodetect already avoids them.

`ports` shows no vendor or product IDs. Expected on the released macOS binaries, which are built without cgo and so cannot read USB descriptors. Identification is unaffected. Build from source natively if you want the IDs.

Linux

`permission denied` opening `/dev/ttyACM1`. Your user is not in the port's group. Add yourself to `dialout` (Debian, Ubuntu, Raspberry Pi OS) or `uucp` (Arch, some Fedora), then log out and back in:

```
sudo usermod -aG dialout "$USER"
```

Confirm with `groups`, and check the port's owner with `ls -l /dev/ttyACM*`.

The first command after plugging in fails, then it works. ModemManager probes new `/dev/ttyACM*` devices for a few seconds, assuming they might be a cellular modem, and holds the port while it does. Install the udev rule in *Installing → Linux* to exempt the board, or if you have no modems at all:

```
sudo systemctl mask ModemManager
```

No `CIRCUITPY` volume when flashing. Many minimal or headless installs do not automount removable media. Find it with `lsblk -o NAME,LABEL,MOUNTPOINT`, mount it, and pass the path:

```
scripts/flash.sh --drive /mnt/CIRCUITPY
```

`kb2040ctl: command not found` after moving it to `/usr/local/bin`. That directory is on the default `PATH` on most distributions but not all; check with `echo $PATH`.